

EVALUATION REPORT

Live Meeting Summarizer Application

Comprehensive Assessment of Functionality, Usability & Performance

Report Type	Academic / Prototype Evaluation
Application	Live Meeting Summarizer
Technology Stack	Python, Streamlit, AI/ML Backend
Report Status	Final
Classification	Confidential

Prepared for academic review and prototype assessment purposes.

1. Introduction

The Live Meeting Summarizer Application is designed to capture meeting audio, generate live captions, produce a full transcript, summarize the discussion, and support export and email sharing. The system combines real-time speech processing with AI-based summarization to reduce manual note-taking effort and improve post-meeting productivity.

This report provides a thorough evaluation of the application across seven key dimensions: functionality, usability, performance, reliability, maintainability, strengths, and scope for improvement. The assessment is intended to identify both the current value delivered by the system and the areas where further development would yield the greatest benefit.

2. Objectives

The main objectives of the project are as follows:

- Record live meeting audio from a selected microphone input device.
- Generate live captions in real time during the meeting.
- Produce a structured, readable transcript after recording ends.
- Generate a concise, AI-powered summary of the meeting content.
- Allow users to export results in Markdown and PDF formats.
- Support sharing of the summary through integrated email functionality.
- Provide a clean, accessible dashboard interface for non-technical users.

3. Evaluation Criteria

The application is evaluated using the following criteria:

Criterion	Rating	Summary Notes
Functional Correctness	Excellent	All core features work as intended end-to-end.
UI & Usability	Good	Clean Streamlit layout; mobile responsiveness needs work.
Performance	Good	Adequate for prototype use; async processing recommended.
Reliability & Error Handling	Good	Protective logic in place; email delivery can be fragile.
Maintainability	Fair	Modular backend; large app.py file a concern.
Practical Usefulness	Excellent	Strong real-world value for meeting capture workflows.

4. Functional Evaluation

4.1 Audio Recording

The application supports microphone-based live recording and allows the user to select from available input devices. This flexibility is important when multiple microphones or audio interfaces are connected to the system.

- Recording control is straightforward through start and stop actions.
- Device selection helps avoid common microphone input conflicts.
- The feature is well-suited for real-time meeting capture scenarios.

4.2 Live Caption Generation

Live captions are generated while recording is in progress. This provides the user with immediate feedback and helps confirm that speech is being captured correctly.

- Real-time captions improve transparency and user confidence during recording.
- Users can quickly detect microphone or input device issues before they escalate.
- Caption accuracy depends on audio quality, speaker clarity, and ambient noise levels.

4.3 Transcript Generation

After recording stops, the application processes the captured audio and produces a structured transcript. Where supported by the backend, speaker labels are displayed to improve readability.

- The transcript is the primary structured output of the system.
- Speaker-labeled formatting significantly improves post-meeting review.
- Transcript quality is directly dependent on speech-to-text backend accuracy and recording conditions.

4.4 Summary Generation

The application produces an AI-generated meeting summary derived from the transcript. This feature directly reduces the effort required to extract key insights from lengthy meetings.

- Summaries reduce the time spent reviewing raw meeting content.
- Useful for quick revision, stakeholder reporting, and follow-up task planning.
- Summary quality degrades when the underlying transcript contains noise or gaps.

4.5 Export and Email Features

Users can export the transcript and summary in Markdown and PDF formats. An integrated email feature allows the summary to be shared directly from the application.

- Export functionality significantly increases the practical usability of outputs.
- PDF sharing makes results suitable for formal communication and archiving.
- Email integration streamlines collaborative post-meeting workflows.

5. User Interface Evaluation

The application uses a Streamlit-based interface with a dashboard-style layout. The UI includes microphone controls, live caption display, session metrics, transcript and summary panels, and sharing options grouped into logical sections.

Positive aspects of the current interface:

- The interface is accessible to non-technical users without prior training.
- The dashboard layout improves readability and task flow progression.
- Status indicators communicate system state (idle, recording, processing, completed) clearly.
- Logical section grouping makes the workflow intuitive to follow.

Areas identified for refinement:

- The UI would benefit from improved mobile responsiveness and adaptive layout behavior.
- Long transcripts could be enhanced with search, speaker filtering, or collapsible sections.
- First-time users would benefit from guided onboarding or contextual help tooltips.
- A session history panel would enhance navigation between past meetings.

6. Performance Evaluation

Performance can be assessed across two distinct stages: live interaction during recording and final processing after recording completes.

6.1 Live Interaction

- Caption updates occur during recording, providing an acceptable real-time experience.
- Responsiveness is suitable for moderate meeting durations and standard meeting sizes.
- Performance may degrade on low-specification hardware or with heavy background processes running concurrently.

6.2 Final Processing

- Transcript and summary generation begins immediately after recording stops.
- Processing time scales with audio duration, model size, and available hardware capability.
- Temporary file creation and cleanup are handled automatically during the processing stage.

Overall, the application is suitable for academic and prototype-level use. For production-scale deployment, asynchronous processing pipelines and hardware acceleration would provide meaningful performance improvements.

7. Reliability and Error Handling

The application incorporates protective logic for several anticipated failure scenarios, including missing microphone input, module import errors, and backend processing failures.

Strengths in the current error handling approach:

- Startup crashes are avoided by deferring some backend module imports.
- Streamlit session state is used effectively to preserve recording state and UI flow.
- User-facing error messages provide meaningful guidance for common failures.

Remaining reliability challenges:

- Email delivery can fail silently due to external SMTP configuration issues.
- Speech recognition accuracy degrades in noisy or acoustically challenging environments.
- Missing or misconfigured backend model dependencies can disable core features without clear diagnosis.
- Network timeouts or model API rate limits are not yet handled gracefully.

8. Maintainability Evaluation

The project structure separates frontend Streamlit behavior from backend processing modules responsible for transcription, export, logging, and email delivery.

Maintainability strengths:

- Modular backend design makes individual components easier to test and replace.
- Shared utility functions for formatting, sanitization, and compatibility improve overall robustness.
- Session state management supports a predictable and reproducible Streamlit workflow.

Maintainability concerns to address:

- A single large app.py file becomes progressively harder to navigate and maintain as the project grows.
- Some UI logic, business logic, and orchestration remain tightly coupled within the same module.
- Absence of automated unit and integration tests creates risk during future refactoring.
- Dependency management (requirements.txt or pyproject.toml) should be reviewed for reproducibility.

9. Strengths

The following strengths distinguish the Live Meeting Summarizer as a well-conceived and practically valuable application:

- Integrates recording, transcription, summarization, export, and email in a single cohesive workflow.
- Live caption generation provides immediate feedback and confidence during recording.
- The Streamlit dashboard offers a clean, accessible interface for non-technical users.
- Practical export outputs (PDF and Markdown) support diverse downstream use cases.
- Modular backend helps promote organized, maintainable code structure.
- Speaker diarization support enhances transcript readability for multi-participant meetings.
- End-to-end coverage of the meeting capture lifecycle sets a strong foundation for further development.

10. Limitations

The following limitations were identified during the evaluation:

- Transcription and summary accuracy are heavily dependent on audio quality and backend model capability.
- Real-time captioning may exhibit slight lag on lower-specification systems.

- Speaker diarization is not always perfectly accurate, particularly for similar-sounding voices.
- The application currently requires local environment setup and dependency installation.
- Advanced collaboration features such as meeting history, search, and team workspace are absent.
- No cloud deployment or multi-user access support is currently available.
- Limited analytics such as speaking-time distribution, sentiment analysis, or decision tracking.

11. Suggested Improvements

The following improvements are recommended for future development iterations:

Improvement Area	Description
Meeting History	Add saved session browsing and history with searchable past transcripts and summaries.
Mobile Responsiveness	Improve adaptive layout behavior for tablet and mobile devices.
Transcript Enhancements	Introduce speaker filters, keyword search, and inline editable notes.
Action Item Extraction	Add a dedicated structured section identifying tasks, owners, and deadlines.
Error Diagnostics	Improve error reporting for microphone failures, model issues, and email errors.
Automated Testing	Add unit and integration tests for all core processing workflows.
Cloud Deployment	Support multi-user access through cloud hosting and authentication.
Meeting Analytics	Add speaking-time distribution, sentiment analysis, and decision tracking.
Plugin Architecture	Enable extensibility for custom summarization models or export formats.
Async Processing	Refactor processing pipelines to use asynchronous operations for better scalability.

12. Overall Assessment

The Live Meeting Summarizer Application is a useful and practically relevant project that successfully addresses a common real-world challenge. By reducing the manual effort required to capture, organize, and summarize meeting content, it delivers genuine value for individual users and small teams alike.

The integration of live captions, transcript generation, AI-powered summarization, export options, and email sharing makes it a complete end-to-end prototype. The project demonstrates clear understanding of applied AI, user interface design, and modular software development practices.

With targeted improvements in performance optimization, test coverage, cloud deployment, and advanced feature development, the application has strong potential to evolve into a production-ready collaborative tool.

13. Conclusion

The evaluation confirms that the Live Meeting Summarizer Application meets its primary objectives and delivers a valuable end-to-end workflow for meeting capture and summarization. While limitations exist in areas of accuracy, dependency management, and scalability, the system is effective as a modern intelligent application and demonstrates meaningful technical depth.

The project can be considered successful, practically useful, and well-positioned for further enhancement. The foundation established by this prototype provides a clear pathway toward a more comprehensive and scalable solution capable of supporting real-world deployment at team or organizational scale.

— End of Report —